

CSS Flexbox Justify-Content

Complete Guide: From Basics to Advanced

Contents

1	Introduction	2
1.1	What is justify-content?	2
1.2	Why Use justify-content?	2
1.3	Main Axis Reminder	2
2	All justify-content Values	3
2.1	Value Overview	3
2.2	Comparison Table	3
3	Core Values	4
3.1	flex-start (Default)	4
3.1.1	Syntax	4
3.1.2	Characteristics	4
3.1.3	Examples	4
3.2	flex-end	5
3.2.1	Syntax	5
3.2.2	Characteristics	5
3.2.3	Examples	6
3.3	center	7
3.3.1	Syntax	7
3.3.2	Characteristics	7
3.3.3	Examples	7
3.4	space-between	9
3.4.1	Syntax	9
3.4.2	Characteristics	9
3.4.3	Examples	9
3.5	space-around	11
3.5.1	Syntax	11
3.5.2	Characteristics	11
3.5.3	Examples	12
3.6	space-evenly	13
3.6.1	Syntax	13
3.6.2	Characteristics	13
3.6.3	Examples	13

4	Combining justify-content with Other Properties	16
4.1	justify-content with align-items	16
4.2	justify-content with gap	16
4.3	justify-content with flex-direction	17
5	Practical Applications	18
5.1	Application 1: Navigation Bar	18
5.2	Application 2: Hero Section	19
5.3	Application 3: Card Grid	20
5.4	Application 4: Footer	21
6	Responsive justify-content	23
6.1	Mobile-First Approach	23
6.2	Responsive Navigation	23
7	Troubleshooting	25
7.1	Issue 1: justify-content Not Working	25
7.2	Issue 2: Direction Wrong	25
7.3	Issue 3: Items Overlapping	26
7.4	Issue 4: Last Row Misaligned	26
8	Best Practices	27
8.1	When to Use Each Value	27
8.2	Common Patterns	27
8.3	Accessibility Tips	28
9	Summary: Key Takeaways	29
9.1	The Seven Values	29
9.2	Key Rules	29
9.3	Quick Reference	30
9.4	Main Axis vs. Cross Axis	30
9.5	Comparison: justify-content Values	31

Chapter 1

Introduction

1.1 What is justify-content?

The `justify-content` property controls how flex items are distributed along the main axis. It defines the alignment of items and the distribution of free space in the flex container.

Key Point: `justify-content` only affects the main axis. The direction of the main axis depends on `flex-direction` (horizontal for row, vertical for column).

1.2 Why Use justify-content?

- Center content without manual calculations
- Distribute items evenly with automatic spacing
- Space items apart with flexible gaps
- Create professional layouts easily
- Responsive alignment without media queries
- Handle dynamic content gracefully
- Cleaner code than margin hacks

1.3 Main Axis Reminder

Remember that `justify-content` always works along the main axis:

- With `flex-direction: row` → Main axis is horizontal
- With `flex-direction: column` → Main axis is vertical
- With `flex-direction: row-reverse` → Main axis is reversed horizontally
- With `flex-direction: column-reverse` → Main axis is reversed vertically

Chapter 2

All justify-content Values

2.1 Value Overview

`justify-content` has seven main values:

1. `flex-start` - Items at the start
2. `flex-end` - Items at the end
3. `center` - Items centered
4. `space-between` - Space between items
5. `space-around` - Space around items
6. `space-evenly` - Equal space everywhere
7. `stretch` - Stretch to fill space

2.2 Comparison Table

Value	Behavior
<code>flex-start</code>	Items at start, remaining space at end
<code>flex-end</code>	Items at end, remaining space at start
<code>center</code>	Items centered, space on both sides
<code>space-between</code>	First/last at edges, space between
<code>space-around</code>	Equal space around each item
<code>space-evenly</code>	Equal space everywhere
<code>stretch</code>	Items grow to fill available space

Chapter 3

Core Values

3.1 flex-start (Default)

Items are packed toward the start of the container.

3.1.1 Syntax

```
.flex-container {  
  display: flex;  
  justify-content: flex-start; /* Default */  
}
```

3.1.2 Characteristics

- Items packed at start of main axis
- Remaining space on the right (row) or bottom (column)
- Default behavior
- No space distribution
- Useful for aligned layouts

3.1.3 Examples

Example 1: Left-Aligned Navigation

```
<nav class="navbar">  
  <a href="#">Home</a>  
  <a href="#">About</a>  
  <a href="#">Contact</a>  
</nav>  
  
.navbar {  
  display: flex;  
  justify-content: flex-start; /* Align left */  
  gap: 20px;  
  background: #2c3e50;
```

```
        padding: 15px;
    }

    .navbar a {
        color: white;
        text-decoration: none;
    }

    /* Output: [Home] [About] [Contact] */
```

Example 2: Sidebar Menu

```
<aside class="sidebar">
    <button>Dashboard</button>
    <button>Settings</button>
    <button>Profile</button>
</aside>

.sidebar {
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    gap: 10px;
    padding: 20px;
}

.sidebar button {
    padding: 12px;
    background: #3498db;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    text-align: left;
}

/* Items aligned to top, space below */
```

3.2 flex-end

Items are packed toward the end of the container.

3.2.1 Syntax

```
.flex-container {
    display: flex;
    justify-content: flex-end;
}
```

3.2.2 Characteristics

- Items packed at end of main axis

- Remaining space at the start
- Useful for right-aligned content
- Works with reversed layouts

3.2.3 Examples

Example 1: Right-Aligned Actions

```
<div class="card-actions">
  <button class="primary">Save</button>
  <button class="secondary">Cancel</button>
</div>

.card-actions {
  display: flex;
  justify-content: flex-end; /* Align right */
  gap: 10px;
  padding: 20px;
}

.card-actions button {
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.card-actions .primary {
  background: #27ae60;
  color: white;
}

.card-actions .secondary {
  background: #ecf0f1;
  color: #333;
}

/* Output:                                [Cancel] [Save] */
```

Example 2: Bottom Right Buttons

```
<div class="modal-footer">
  <button class="delete">Delete</button>
  <button class="confirm">Confirm</button>
</div>

.modal-footer {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
  padding: 15px;
  border-top: 1px solid #ddd;
}
```



```
/* Buttons appear at bottom right */
```

3.3 center

Items are centered in the container.

3.3.1 Syntax

```
.flex-container {  
  display: flex;  
  justify-content: center;  
}
```

3.3.2 Characteristics

- Items centered on main axis
- Equal space on both sides
- Most intuitive for centering
- Works with any flex-direction
- Perfect for symmetric layouts

3.3.3 Examples

Example 1: Centered Header

```
<header class="page-header">  
  <h1>Welcome</h1>  
</header>  
  
.page-header {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 200px;  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
}  
  
.page-header h1 {  
  color: white;  
  margin: 0;  
  font-size: 48px;  
}  
  
/* Heading centered perfectly */
```

Example 2: Centered Logo

```
<div class="logo-container">
  
</div>

.logo-container {
  display: flex;
  justify-content: center;
  padding: 40px;
}

.logo {
  width: 100px;
  height: 100px;
}

/* Logo centered horizontally */
```

Example 3: Centered Button

```
<div class="button-container">
  <button class="cta">Get Started</button>
</div>

.button-container {
  display: flex;
  justify-content: center;
  padding: 40px;
}

.cta {
  padding: 15px 40px;
  background: #3498db;
  color: white;
  border: none;
  border-radius: 4px;
  font-size: 16px;
  cursor: pointer;
}

/* Button centered */
```

Example 4: Centered Icons

```
<div class="icon-row">
  <span class="icon "></span>
  <span class="icon "></span>
  <span class="icon "></span>
</div>

.icon-row {
  display: flex;
  justify-content: center;
  gap: 30px;
  padding: 40px;
  font-size: 48px;
}
```

```
}  
  
/* Icons centered with gaps */
```

3.4 space-between

First item at start, last item at end, remaining space distributed between.

3.4.1 Syntax

```
.flex-container {  
    display: flex;  
    justify-content: space-between;  
}
```

3.4.2 Characteristics

- First item at container start
- Last item at container end
- Remaining items distributed evenly
- Excellent for navigation bars
- Perfect for space distribution

3.4.3 Examples

Example 1: Navigation with Logo and Links

```
<nav class="navbar">  
    <div class="logo">Logo</div>  
    <div class="nav-center">  
        <a href="#">Home</a>  
        <a href="#">Products</a>  
        <a href="#">Services</a>  
    </div>  
    <button class="login">Login</button>  
</nav>  
  
.navbar {  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
    gap: 20px;  
    padding: 15px 30px;  
    background: #2c3e50;  
}  
  
.logo {  
    color: white;
```

```
        font-weight: bold;
    }

    .nav-center {
        display: flex;
        gap: 20px;
    }

    .nav-center a {
        color: white;
        text-decoration: none;
    }

    .login {
        background: #3498db;
        color: white;
        border: none;
        padding: 8px 16px;
        border-radius: 4px;
        cursor: pointer;
    }

    /* Logo left, links center, login right */
```

Example 2: Header with Info

```
<div class="page-header">
  <h2>Page Title</h2>
  <span class="last-updated">Updated: 2024-01-15</span>
</div>

.page-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 20px;
    border-bottom: 1px solid #ddd;
}

.page-header h2 {
    margin: 0;
}

.last-updated {
    color: #666;
    font-size: 12px;
}

/* Title on left, timestamp on right */
```

Example 3: Card Header with Actions

```
<div class="card-header">
  <h3>My Card</h3>
  <div class="actions">
```

```
        <button>Edit</button>
        <button>Delete</button>
    </div>
</div>

.card-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 15px;
}

.card-header h3 {
    margin: 0;
}

.actions {
    display: flex;
    gap: 8px;
}

.actions button {
    padding: 6px 12px;
    background: #ecf0f1;
    border: none;
    cursor: pointer;
}

/* Title left, actions right */
```

3.5 space-around

Equal space distributed around each item (space on both sides of each item).

3.5.1 Syntax

```
.flex-container {
    display: flex;
    justify-content: space-around;
}
```

3.5.2 Characteristics

- Each item has equal space on both sides
- First and last items have half-space at edges
- Visually centered items with gaps
- Creates even visual rhythm
- Good for symmetric layouts

3.5.3 Examples

Example 1: Feature Icons

```
<div class="features">
  <div class="feature">
    <div class="icon "></div>
    <p>Fast</p>
  </div>
  <div class="feature">
    <div class="icon "></div>
    <p>Secure</p>
  </div>
  <div class="feature">
    <div class="icon "></div>
    <p>Beautiful</p>
  </div>
</div>

.features {
  display: flex;
  justify-content: space-around;
  padding: 40px 20px;
}

.feature {
  text-align: center;
}

.feature .icon {
  font-size: 48px;
  margin-bottom: 15px;
}

/* Icons evenly spaced with room around each */
```

Example 2: Social Media Links

```
<div class="social-links">
  <a href="#" class="social">f</a>
  <a href="#" class="social">t</a>
  <a href="#" class="social">ig</a>
  <a href="#" class="social">in</a>
</div>

.social-links {
  display: flex;
  justify-content: space-around;
  max-width: 400px;
}

.social {
  width: 40px;
  height: 40px;
  background: #3498db;
```

```
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 50%;
    text-decoration: none;
    font-weight: bold;
}

/* Social icons evenly distributed */
```

3.6 space-evenly

Equal space distributed everywhere (between items and at edges).

3.6.1 Syntax

```
.flex-container {
    display: flex;
    justify-content: space-evenly;
}
```

3.6.2 Characteristics

- Equal space between all items
- Equal space at start and end edges
- Most even distribution
- Perfect symmetry
- Modern and professional look

3.6.3 Examples

Example 1: Pricing Tiers

```
<div class="pricing-tiers">
  <div class="tier">
    <h3>Basic</h3>
    <p class="price">$9</p>
    <button>Choose</button>
  </div>
  <div class="tier">
    <h3>Pro</h3>
    <p class="price">$29</p>
    <button>Choose</button>
  </div>
  <div class="tier">
    <h3>Enterprise</h3>
    <p class="price">$99</p>
```

```
        <button>Choose</button>
    </div>
</div>

.pricing-tiers {
    display: flex;
    justify-content: space-evenly;
    gap: 20px;
    padding: 40px;
    flex-wrap: wrap;
}

.tier {
    flex: 1 1 200px;
    background: white;
    padding: 30px;
    border-radius: 8px;
    text-align: center;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

.tier h3 {
    margin-top: 0;
}

.tier .price {
    font-size: 32px;
    font-weight: bold;
    color: #3498db;
    margin: 20px 0;
}

/* Tiers evenly spaced */
```

Example 2: Stat Boxes

```
<div class="stats">
    <div class="stat">
        <div class="number">1000+</div>
        <div class="label">Users</div>
    </div>
    <div class="stat">
        <div class="number">50K</div>
        <div class="label">Downloads</div>
    </div>
    <div class="stat">
        <div class="number">4.9/5</div>
        <div class="label">Rating</div>
    </div>
</div>

.stats {
    display: flex;
    justify-content: space-evenly;
    padding: 40px;
}
```



```
        background: #f8f9fa;
    }

    .stat {
        text-align: center;
    }

    .stat .number {
        font-size: 32px;
        font-weight: bold;
        color: #3498db;
    }

    .stat .label {
        color: #666;
        margin-top: 8px;
    }

    /* Stats perfectly centered and spaced */
```

Chapter 4

Combining justify-content with Other Properties

4.1 justify-content with align-items

For perfect alignment in both directions:

```
/* Perfect centering (most common) */
.perfect-center {
  display: flex;
  justify-content: center; /* Horizontal center */
  align-items: center; /* Vertical center */
  width: 400px;
  height: 400px;
}

/* Spread horizontally, centered vertically */
.spread-centered {
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 100px;
}

/* Clustered center, spread vertically */
.clustered-spread {
  display: flex;
  flex-direction: column;
  justify-content: space-around;
  align-items: center;
  height: 300px;
}
```

4.2 justify-content with gap

The gap property works independently of justify-content:

```
/* gap adds fixed space, justify-content distributes remainder */
```

```
.container {
  display: flex;
  justify-content: space-between;
  gap: 20px; /* Fixed gap between items */
}

/* gap with centered items */
.centered-with-gap {
  display: flex;
  justify-content: center;
  gap: 15px;
}

/* gap + space-around creates extra spacing */
.spaced {
  display: flex;
  justify-content: space-around;
  gap: 20px;
}
```

4.3 justify-content with flex-direction

Remember that justify-content direction changes with flex-direction:

```
/* Row: justify-content affects horizontal */
.horizontal-center {
  display: flex;
  flex-direction: row;
  justify-content: center; /* Horizontal center */
}

/* Column: justify-content affects vertical */
.vertical-center {
  display: flex;
  flex-direction: column;
  justify-content: center; /* Vertical center */
}

/* Row-reverse: justify-content direction reversed */
.rtl-center {
  display: flex;
  flex-direction: row-reverse;
  justify-content: center; /* Still centers on main axis */
}
```

Chapter 5

Practical Applications

5.1 Application 1: Navigation Bar

```
<header class="header">
  <nav class="navbar">
    <div class="logo">MyApp</div>
    <div class="nav-links">
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Services</a>
      <a href="#">Contact</a>
    </div>
    <div class="user-menu">
      <button class="login">Login</button>
      <button class="signup">Sign Up</button>
    </div>
  </nav>
</header>

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px 30px;
  background: #2c3e50;
}

.logo {
  color: white;
  font-weight: bold;
  font-size: 20px;
}

.nav-links {
  display: flex;
  justify-content: center;
  gap: 30px;
}
```

```
.nav-links a {
  color: white;
  text-decoration: none;
}

.user-menu {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
}

.user-menu button {
  padding: 8px 16px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.login {
  background: transparent;
  color: white;
}

.signup {
  background: #3498db;
  color: white;
}

/* Logo left, links center, buttons right */
```

5.2 Application 2: Hero Section

```
<section class="hero">
  <div class="hero-content">
    <h1>Welcome to Our Service</h1>
    <p>Build amazing things</p>
    <div class="hero-actions">
      <button class="primary">Get Started</button>
      <button class="secondary">Learn More</button>
    </div>
  </div>
</section>

.hero {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 500px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  text-align: center;
}
```

```
.hero-content {
  color: white;
  max-width: 600px;
}

.hero-content h1 {
  font-size: 48px;
  margin: 0 0 20px 0;
}

.hero-content p {
  font-size: 24px;
  margin: 0 0 30px 0;
}

.hero-actions {
  display: flex;
  justify-content: center;
  gap: 15px;
}

.hero-actions button {
  padding: 12px 30px;
  border: none;
  border-radius: 4px;
  font-size: 16px;
  cursor: pointer;
}

.primary {
  background: white;
  color: #667eea;
}

.secondary {
  background: transparent;
  color: white;
  border: 2px solid white;
}

/* All content centered both ways */
```

5.3 Application 3: Card Grid

```
<div class="card-grid">
  <div class="card">
    <h3>Feature 1</h3>
    <p>Description here</p>
  </div>
  <div class="card">
    <h3>Feature 2</h3>
    <p>Description here</p>
```

```

    </div>
    <div class="card">
      <h3>Feature 3</h3>
      <p>Description here</p>
    </div>
  </div>
</div>

.card-grid {
  display: flex;
  justify-content: space-around;
  flex-wrap: wrap;
  gap: 20px;
  padding: 40px;
}

.card {
  flex: 1 1 250px;
  background: white;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
  text-align: center;
}

.card h3 {
  margin-top: 0;
}

/* Cards evenly distributed */

```

5.4 Application 4: Footer

```

<footer class="footer">
  <div class="footer-section">
    <h4>Company</h4>
    <ul>
      <li><a href="#">About</a></li>
      <li><a href="#">Careers</a></li>
      <li><a href="#">Blog</a></li>
    </ul>
  </div>
  <div class="footer-section">
    <h4>Legal</h4>
    <ul>
      <li><a href="#">Privacy</a></li>
      <li><a href="#">Terms</a></li>
      <li><a href="#">Cookies</a></li>
    </ul>
  </div>
  <div class="footer-section">
    <h4>Social</h4>
    <ul>

```

```
        <li><a href="#">Twitter</a></li>
        <li><a href="#">LinkedIn</a></li>
        <li><a href="#">GitHub</a></li>
    </ul>
</div>
</footer>

.footer {
    display: flex;
    justify-content: space-around;
    gap: 40px;
    padding: 40px;
    background: #2c3e50;
    color: white;
}

.footer-section h4 {
    margin-top: 0;
}

.footer-section ul {
    list-style: none;
    padding: 0;
    margin: 0;
}

.footer-section li {
    margin: 10px 0;
}

.footer-section a {
    color: #bdc3c7;
    text-decoration: none;
}

/* Sections evenly spaced */
```


Chapter 6

Responsive justify-content

6.1 Mobile-First Approach

```
/* Mobile: Stack vertically, center */
.layout {
  display: flex;
  flex-direction: column;
  justify-content: center;
}

/* Tablet: Horizontal, space between */
@media (min-width: 768px) {
  .layout {
    flex-direction: row;
    justify-content: space-between;
  }
}

/* Desktop: Still space between, more gap */
@media (min-width: 1200px) {
  .layout {
    gap: 40px;
  }
}
```

6.2 Responsive Navigation

```
/* Mobile: Stacked, centered */
.navbar {
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 10px;
}

/* Tablet: Horizontal, space-between */
@media (min-width: 768px) {
```

```
.navbar {  
  flex-direction: row;  
  justify-content: space-between;  
  align-items: center;  
}  
}
```

Chapter 7

Troubleshooting

7.1 Issue 1: justify-content Not Working

Problem: justify-content seems to have no effect.

Solution: Ensure parent has display: flex.

```
/* WRONG: No flex display */
.container {
  justify-content: center; /* Does nothing */
}

/* CORRECT: Add display: flex */
.container {
  display: flex;
  justify-content: center; /* Works now */
}
```

7.2 Issue 2: Direction Wrong

Problem: Items aligning wrong direction.

Solution: Check flex-direction - it controls which axis justify-content affects.

```
/* Row: justify-content is horizontal */
.horizontal {
  display: flex;
  flex-direction: row;
  justify-content: center; /* Horizontal center */
}

/* Column: justify-content is vertical */
.vertical {
  display: flex;
  flex-direction: column;
  justify-content: center; /* Vertical center */
}
```

7.3 Issue 3: Items Overlapping

Problem: Items overlap with justify-content.

Solution: Add gap property or reduce item size.

```
.container {  
  display: flex;  
  justify-content: space-between;  
  gap: 20px; /* Add gap */  
}
```

7.4 Issue 4: Last Row Misaligned

Problem: With flex-wrap, last row aligns differently.

Solution: Use justify-content: flex-start for consistent alignment.

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
  justify-content: flex-start; /* Consistent alignment */  
  gap: 15px;  
}
```

Chapter 8

Best Practices

8.1 When to Use Each Value

- **flex-start:** Left-aligned layouts, navigation
- **flex-end:** Right-aligned buttons, RTL languages
- **center:** Centered headers, symmetric layouts
- **space-between:** Navigation bars, headers with info
- **space-around:** Icon displays, feature showcases
- **space-evenly:** Stat boxes, pricing tiers

8.2 Common Patterns

```
/* Perfect centering */
.center {
  display: flex;
  justify-content: center;
  align-items: center;
}

/* Space between (nav pattern) */
.spread {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

/* Centered with gap */
.centered-gap {
  display: flex;
  justify-content: center;
  gap: 20px;
}

/* Evenly distributed */
```

```
.even {  
  display: flex;  
  justify-content: space-evenly;  
  flex-wrap: wrap;  
}
```

8.3 Accessibility Tips

- Maintain logical reading order
- Don't use `space-between` to hide content
- Ensure sufficient spacing for touch targets (at least 44px)
- Test keyboard navigation
- Consider screen reader experience

Chapter 9

Summary: Key Takeaways

9.1 The Seven Values

Value	Best For
flex-start	Left/top alignment
flex-end	Right/bottom alignment
center	Centered content
space-between	Space distribution (nav bars)
space-around	Even distribution with edges
space-evenly	Perfect symmetry
stretch	Growing to fill space

9.2 Key Rules

1. Works only on main axis
2. Main axis depends on flex-direction
3. Requires `display: flex` to work
4. Works great with align-items for 2D alignment
5. Combine with gap for cleaner code
6. Can be changed with media queries
7. Responsive without explicit breakpoints
8. Most common: center, space-between, space-around
9. Combine with flex for flexible items
10. Test at different screen sizes

9.3 Quick Reference

```
/* Center everything */
.center-all {
  display: flex;
  justify-content: center;
  align-items: center;
}

/* Navigation pattern */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

/* Spread evenly */
.spread {
  display: flex;
  justify-content: space-evenly;
  flex-wrap: wrap;
}

/* Left align (default) */
.left {
  display: flex;
  justify-content: flex-start;
}

/* Right align */
.right {
  display: flex;
  justify-content: flex-end;
}

/* With gap */
.with-gap {
  display: flex;
  justify-content: center;
  gap: 20px;
}
```

9.4 Main Axis vs. Cross Axis

- **justify-content:** Aligns on main axis
- **align-items:** Aligns on cross axis
- Main axis direction depends on flex-direction
- Both needed for complete 2D alignment
- Together create professional layouts

9.5 Comparison: justify-content Values

flex-start	center	flex-end	space-*
[Items]...	...[Items]..	...[Items]	[I]..[I]..[I]